Auditable Failure Memory for ROS 2 Navigation: An Exploratory Comparison with Spatial Caching

Author One* Author Two† Author Three‡

Abstract

In multi-phase indoor robot navigation, reactive navigation systems lacking cross-phase memory repeatedly re-enter occluded dead-end corridors across sequential tasks when initial line-of-sight is obstructed. We present **FailMem**, an event-driven failure memory architecture for ROS 2 Nav2 that binds navigation aborts to immutable observation bundles—combining raw LiDAR scans, coordinate transforms, and local costmap regions of interest—and provides perception-driven dynamic invalidation when clearance is verified by sensors. Within an asymmetric dual-path environment in Gazebo 11 simulations with a TurtleBot3 robot, we evaluate FailMem across 30 Gazebo simulation runs ($n = 3$ per condition under sequential fixed seeds) against reactive retry (R), persistent suppression ($M1$), and spatial observation caching (O) baselines, alongside an exploratory 4-run feasibility trial on action-conditioned navigation (H_1). Memory retention and invalidation are evaluated across sequential phases within a single continuous runner execution session. The evaluation demonstrates that cross-phase memory successfully prevents redundant dead-end exploration in obstructed settings, and dynamic invalidation unsuppresses the shorter route upon blockage removal (reducing travel distance by $\approx 11.0\%$ relative to permanent suppression, though total episode duration showed no observed improvement). Under static 2D geometry, FailMem and spatial caching chose identical topological routes with under 1% difference in distance and duration, showing no additional routing benefit of FailMem over spatial caching in the evaluated layout and run sample. Furthermore, in an exploratory 4-run feasibility trial on action-conditioned navigation (H_1), Nav2 returned success in 4/4 runs, with strict physical arrival verified in 3/4 runs; this candidate scenario did not establish the expected executability divergence, leading to stopping per protocol.

1 Introduction

Autonomous mobile robots operating in structured indoor facilities—such as warehouses, hospitals, and office corridors—frequently encounter unmapped or transient physical blockages (such as parked delivery carts or closed safety doors) along nominal routes. Standard navigation systems, including the ROS 2 Navigation Stack (Nav2) [1,2], utilize layered 2D costmaps [3] and local recovery behaviors (e.g., in-place rotations, backup maneuvers, costmap clearing). In standard Nav2 configurations, navigation actions are dispatched independently, and recovery routines operate within the lifetime of an individual goal execution [1]. Consequently, when goals are dispatched sequentially from a distant decision junction without cross-goal memory, a purely reactive agent re-enters known dead ends until on-board sensors re-acquire line-of-sight to the blockage.

To mitigate unguided re-exploration, two primary approaches exist:

1. **Spatial Costmap Caching (O)**: Maintaining geometric occupancy in persistent global grids (e.g., Costmap2D [3], OctoMap [4], Voxblox [5]) as sensory observations accumulate.

2. **Episodic Failure Memory (F)**: Recording explicit execution aborts, error causes, or symbolic constraints to suppress infeasible route options in high-level dispatching.

While embodied AI frameworks explore language-based reflection (e.g., Reflexion [6], REFLECT [7]), they are designed for high-level semantic task planning and verbal reflection. In mobile robot navigation, where spatial occupancy is continuously tracked in metric grids, integrating discrete execution failures with sensor-verified invalidation requires explicit coordination with perception. This raises an empirical question: *Under what conditions does an autonomous mobile robot genuinely require explicit episodic failure memory over standard spatial costmap caching?*

In this work, we present **FailMem**, an event-driven failure memory architecture for ROS 2 Nav2, and conduct an exploratory empirical study to delineate its operational boundaries. Specifically, we provide:

1. **Perception-Grounded Event Memory and Dynamic Invalidation Implementation**: A ROS 2 Nav2 architecture that binds execution aborts to immutable observation bundles (raw LiDAR scans, TF transforms, costmap ROIs) and executes sensor-verified dynamic invalidation upon clearing.

*Department of Robotics, University Placeholder. Correspondence to: author.one@example.edu

†Department of Computer Science, Institute Placeholder.

‡Autonomous Systems Laboratory, University Placeholder.

2. **Auditable Exploratory Navigation Comparison**: A reproducible 30-run benchmark ($n = 3$ per condition under sequential fixed seeds) in Gazebo 11 simulation runs comparing FailMem (F) against reactive (R), spatial caching (O), and persistent suppression ($M1$) baselines across clean (D0), blocked (D1), and restored (D2) scenarios.

3. **Boundary Findings on Failure Memory vs. Spatial Caching**: An empirical finding that within the evaluated static 2D hallway layout, specific implementation, and run sample ($n = 3$), FailMem yields no observed routing benefit over spatial caching (O), selecting identical topological routes ($< 1\%$ difference in distance and duration); alongside empirical stopping criteria from an exploratory 4-run feasibility trial where action-conditioned navigation (H_1) did not establish executability divergence (Nav2 returned success in 4/4 runs, with strict physical arrival verified in 3/4 runs, without establishing the expected executability divergence).

2 Related Work & Mechanism Taxonomy

2.1 Spatial Occupancy Mapping & Distance Fields

Layered 2D costmaps [1,3] maintain geometric obstacles by ray-casting sensor ranges into grid cells and propagating configuration-space inflation. 3D volumetric frameworks, such as OctoMap [4], employ hierarchical octrees with discrete probabilistic voxel occupancy updates. Distance-field frameworks such as Voxblox [5] incrementally construct continuous Euclidean Signed Distance Fields (ESDF) for smooth trajectory optimization. These mapping representations are fundamentally designed to track geometric occupancy and distance fields from sensor rays. Because their core primitive is geometric spatial state rather than execution history, they do not inherently index episodic navigation action aborts or high-level failure metadata.

2.2 Execution Monitoring & Plan Diagnosis

Deliberative robotics frameworks [8–10] formalize plan execution monitoring and diagnostic repair. Modern navigation stacks, including Nav2 [1], execute local recovery behaviors (such as in-place rotations, costmap clearing, or backup maneuvers) during active trajectory tracking. In standard Nav2 configurations, navigation actions are dispatched independently, and recovery routines operate within the scope of an individual goal execution [1]. Extending navigation to retain information across sequential goal dispatches can be addressed either through persistent global occupancy layers or via dedicated episodic failure stores at the decision layer.

2.3 LLM Self-Reflection & Agent Memory

Recent embodied AI systems investigate large language models for planning and error correction. Reflexion [6] prompts LLMs with scalar task feedback to generate natural language self-reflections across trials. Inner Monologue [11] integrates success detectors and scene descriptions into closed-loop prompt chains. REFLECT [7] uses hierarchical multi-modal experience summarization with LLMs to generate structured failure explanations and plan corrections. While these frameworks demonstrate prompt-level flexibility over semantic descriptions, mobile robot navigation also requires grounding execution outcomes in metric sensor coordinates and spatial costmaps. FailMem addresses this complementary domain by investigating event-driven failure recording, perception-verified rule invalidation, and offline replay auditing directly within the ROS 2 Nav2 action stack.

2.4 Experience-Based Planning & No-Good Learning

In motion planning, the Lightning framework [12] and Experience Graphs (E-Graphs) [13] reuse libraries of prior successful paths to accelerate search. In discrete optimization, conflict-driven clause learning (e.g., GRASP [14]) records "no-goods" from search dead ends to prune infeasible branches. FailMem adapts the principle of negative constraint recording to mobile robot navigation with explicit sensor-verified invalidation.

Table 1 summarizes memory representations and state update/invalidation concepts across navigation and reasoning paradigms. Importantly, state update, ray clearing, graph invalidation, and symbolic rule expiration represent conceptually distinct mechanisms across paradigms: geometric mappers update cell occupancy via ray casting, path libraries perform collision checks during repair, LLM agents reset context across trials, while FailMem introduces perception-verified rule invalidation for recorded failure events.

3 The FailMem Architecture

Figure 1 illustrates the overall system architecture, encompassing the execution runtime, immutable observation bundles, and memory lifecycle.

3.1 Immutable Observation Bundles

Prior to evaluating passage clearance, FailMem constructs an immutable observation bundle B:

$$B = \langle \text{obs_id}, \mathbf{z}_{\text{scan}}, \mathbf{T}_{\text{map}\to\text{base}}, \mathbf{M}_{\text{costmap_ROI}}, t_{\text{msg}}, t_{\text{capture}}, t_{\text{eval}} \rangle \quad (1)$$

Table 1: Taxonomy of Memory Representations and State Update/Invalidation Concepts in Robot Navigation and Embodied Reasoning.

Paradigm	Representation	Grounding Level	State Update / Invalidation Concept
Costmap2D / Nav2 [3]	2D Occupancy Grid	Geometric cells	Ray-clearing through free space
OctoMap [4]	3D Voxel Octree	Discrete 3D voxels	Ray-casting probabilistic update
Voxblox [5]	Continuous ESDF / TSDF	3D Distance Field	Dynamic voxel weight integration
Experience Graphs [13]	Path Waypoints / Graph	Discrete Path	Edge collision check during repair
Reflexion [6]	Natural Language Text	Semantic tokens	Trial reset / verbal memory update
REFLECT [7]	Natural Language Text	Multi-modal logs	Task execution cycle reset
FailMem (Implemented)	Event Tuple	Raw sensor bundle	Sensor-verified `FREE` classification

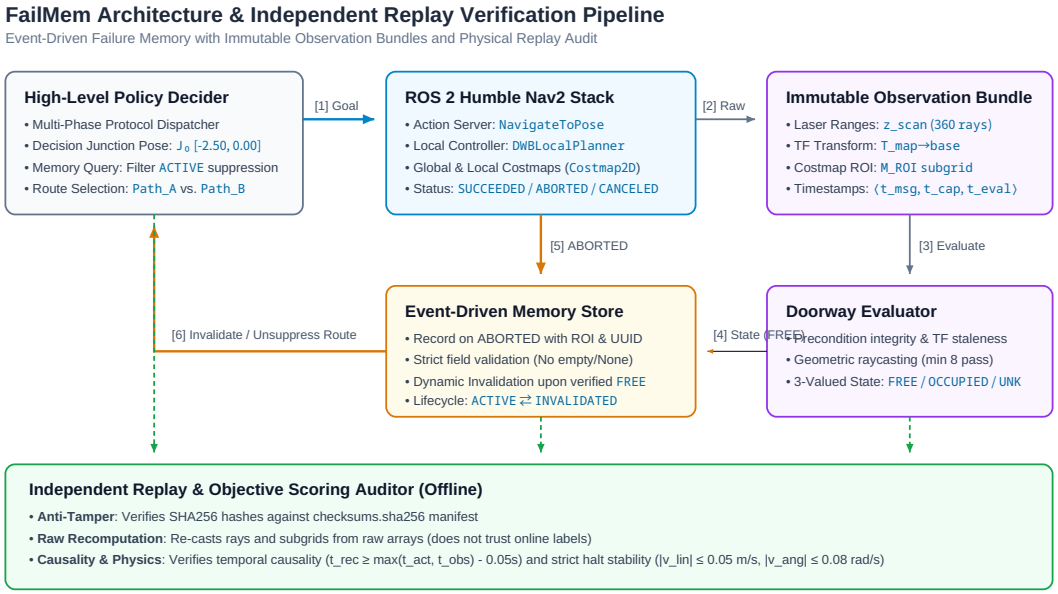


Figure 1: FailMem system architecture and verification pipeline, encompassing execution runtime, immutable observation bundles, and memory lifecycle.

where $\mathbf{z}_{\text{scan}}$ is the raw range array, $\mathbf{T}_{\text{map}\to\text{base}}$ is the TF transformation, and $\mathbf{M}_{\text{costmap_ROI}}$ is the extracted 2D subgrid matrix.

3.2 Clearance Classification Rules

The passage state is evaluated according to strict geometric criteria implemented in `src/doorway_evaluator.py`:

$$\text{State}(B) = \begin{cases} \text{OCCUPIED}, & \text{if } N_{\text{hits_inside}} \geq 5 \\ & \quad \vee\, N_{\text{costmap_occ}} > 0 \\ \text{FREE}, & \text{if } N_{\text{hits_inside}} = 0 \\ & \quad \wedge\, N_{\text{pass_through}} \geq 8 \\ & \quad \wedge\, \text{CostmapCleared}(B) \\ \text{UNKNOWN}, & \text{otherwise} \end{cases} \tag{2}$$

where $N_{\text{costmap_occ}}$ is the count of occupied costmap cells, and $\text{CostmapCleared}(B) \iff (N_{\text{unknown_cells}} = 0 \wedge N_{\text{costmap_occ}} = 0)$ within the doorway ROI $[-0.15, 0.15, 0.95, 1.45]$. (See Supplementary Material S1 for detailed transform alignment and ray projection preconditions).

3.3 Memory Lifecycle & Invalidation Mechanics

- **Failure Registration**: When navigation fails within an occupied passage, an active failure record is created:

$$M = \langle \text{mem_id}, t_{\text{fail}}, \text{goal_uuid}, \\ \text{action_id}, \text{region_id}, \mathbf{M}_{\text{costmap_ROI}} \rangle \tag{3}$$

  Active failure records suppress the associated route at Decision Junction J_0.

- **Dynamic Invalidation**: When subsequent sensor perception yields $\text{State}(B) = \text{FREE}$ with evidence timestamp $t_{\text{free}} > t_{\text{fail}}$, the memory entry is marked INVALIDATED, unsuppressing the route.

- **Cross-Phase Scope Clarification**: In the current experimental implementation, history collection, memory creation, decision dispatch, and memory reset occur within a single continuous runner session across sequential phases. This evaluates phase-to-phase memory transfer within an experimental run, and has not yet been benchmarked as a persistent multi-task database across separate OS daemon lifetimes.

- **Evaluation Pipeline**: An independent replay auditor verifies all serialized logs against SHA256 checksum manifests (detecting file changes relative to the manifest) and physical halt stability criteria ($|v_{\text{lin}}| \leq 0.05\,\text{m/s}, |v_{\text{ang}}| \leq 0.08\,\text{rad/s}$) without reliance on self-reported runtime assertions (detailed in Supplementary Material Section S2).

4 Experimental Setup & Benchmarking Protocol

4.1 Gazebo Simulation Environment & Geometry

Simulations are conducted in ROS 2 Humble with Gazebo 11 simulating a TurtleBot3 Waffle robot in an asymmetric dual-path environment (`configs/p2c_dualpath_world.model`).

- **Path A (North Short Path)**: Waypoint polyline length is 6.124 m from Decision Junction $J_0[-2.50, 0.00]$ to Goal $[2.50, 0.00]$ through a 0.80 m doorway chokepoint at $y = 1.20$.

- **Path B (South Bypass Detour)**: Waypoint polyline length is 9.105 m from J_0 to Goal via open corridor at $y = -2.40$.

4.2 Distance Metric Definitions

To evaluate performance across different phases, we define:

1. **Decision Distance (d_{dec})**: Physical odometry distance traversed during Phase 2 after goal dispatch from Decision Junction J_0 (Path B direct: ≈ 7.54 m; Path A direct: ≈ 5.89 m).

2. **Total Distance (d_{tot})**: Cumulative odometry distance spanning both Phase 1 (historical observation/attempt) and Phase 2 (dispatch to goal).

4.3 Evaluated Scenarios & Baseline Methods

- **Scenario D0 (Unobstructed)**: Doorway is clear; robot navigates Path A without prior history.

- **Scenario D1 (Blocked)**: Doorway chokepoint is obstructed by a physical obstacle box. Phase 1 executes the following sequence: (i) *Vantage Navigation*: The robot navigates from Decision Junction $J_0[-2.50, 0.00]$ to approach vantage waypoint $[-1.00, 1.20, 0.0]$ and captures initial sensor observation bundle; (ii) *Chokepoint Traversal Attempt*: The robot dispatches a genuine Nav2 action attempting chokepoint traversal to $[0.50, 1.20, 0.0]$ with a 15.0 s timeout; (iii) *Action Termination*: Nav2 execution aborts due to physical obstacle obstruction; (iv) *Failure Memory Creation*: Upon action termination, the robot captures a fresh post-traversal observation bundle confirming `OCCUPIED`, classifies the failure cause, and commits an active failure memory record (or updates spatial cache O); (v) *Structured Return*: The robot executes a return navigation action back to Decision Junction J_0. Phase 2 then evaluates routing dispatched from J_0. Under Reactive R, having no cross-phase memory, the robot dispatches the nominal shortest path (Path A), physically travels to entrance waypoint $[-1.50, 1.20]$, detects the blockage on live sensors, and retreats to J_0 before taking Path B. This decision-phase unguided probe in R is strictly distinct from the historical Phase 1 traversal attempt executed in F and O.

- **Scenario D2 (Cleared)**: Following D1, the obstacle is removed. Phase 1 observes `FREE` doorway clearance from vantage waypoint $[-1.00, 1.20]$ and retreats to J_0. Phase 2 evaluates routing from J_0.

Evaluated Methods:

1. **Reactive Baseline (R)**: No cross-phase memory. In D1, enters Path A approach leg, detects blockage at $[-1.50, 1.20]$, and retreats to J_0 before taking Path B.

2. **Spatial Observation Cache (O)**: Caches 2D geometric occupancy observed from vantage point during Phase 1. Evaluated at J_0 during Phase 2.

3. **FailMem (F)**: Event-driven failure memory with dynamic perception-driven invalidation.

4. **Persistent Suppression ($M1$)**: Static failure memory permanently suppressing Path A without dynamic invalidation.

All runs use $n = 3$ per condition under sequential fixed seeds with an episode simulation time budget of 180.0 s, yielding 30 Gazebo simulation runs in total.

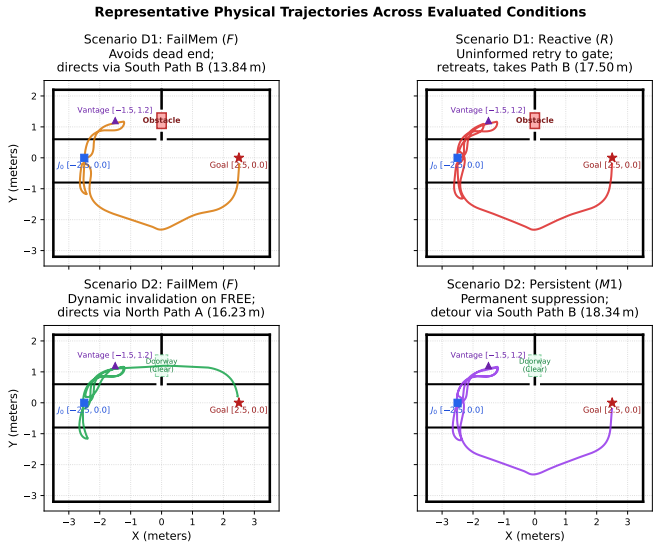


Figure 2: Representative Gazebo simulation trajectories across evaluated conditions.

5 Empirical Results & Boundary Analysis

5.1 Condition-Level Performance

Table 2 presents condition-level performance across 30 Gazebo simulation runs ($n = 3$ per condition, reporting mean ± sample standard deviation with Bessel correction $ddof = 1$). All 10 conditions achieved a 100% replay audit pass rate (3/3), confirming physical halt stability and data completeness across all runs.

5.2 Pairwise Contrasts & Trajectory Analysis

Table 3 reports pairwise contrast comparisons across evaluated methods, and Figure 2 illustrates representative trajectories from the Gazebo simulation runs.

1. **Elimination of Redundant Dead-End Exploration (F vs. R)**: In Scenario D1, FailMem (F) eliminates redundant entry into the blocked corridor (0.0 vs. 1.0 dead ends), reducing total distance by −18.5% (14.03 m vs. 17.22 m) and simulation duration by −22.3% (109.5 s vs. 141.0 s) relative to Reactive R (Table 3, Figure 2). Reactive R's dead-end traversal represents an unguided retry to the entrance waypoint, not collision.

2. **Prevention of Detour Overhead (F vs. $M1$)**: In Scenario D2, dynamic invalidation upon observing `FREE` unsuppresses Path A, reducing total distance by ≈ 11.0% (2.00 m, 16.25 m vs. 18.25 m) relative to persistent suppression ($M1$). However, in the current 3-run exploratory evaluation, average total simulation duration showed no observed improvement and increased slightly by ≈ 1.5% (151.1 s vs. 148.8 s, a sample mean difference of +2.27 s). An independent three-phase trajectory time ledger analysis reveals the empirical source of this discrepancy: the +2.27 s full-episode duration sample mean difference was accrued predominantly during the **History Exploration Phase** ($t < T_{\text{history}}$), where FailMem took 100.00 ± 7.72 s vs. $M1$'s 97.53 ± 2.12 s (a +2.47 s exploration sample mean difference accrued prior to route dispatching). During the **Decision Phase** ($t \geq T_{\text{history}}$), execution durations between FailMem and $M1$ were virtually identical (51.10 ± 0.95 s vs. 51.30 ± 2.00 s, a difference of only −0.20 s), indicating no observable end-to-end time advantage for Path A in this benchmark. An exploratory kinematic decomposition shows that along Path A, active moving time was shorter (28.70 ± 1.18 s vs. 34.30 ± 1.76 s at default thresholds, a sample mean difference of −5.60 s), reflecting shorter path length (5.89 m vs. 7.89 m), while stopped time was longer (15.83 ± 2.42 s vs. 12.13±1.41 s, a sample mean difference of +3.70 s), alongside independent doorway clearance settling window measurements (6.97 ± 0.06 s vs. 4.60 ± 0.00 s). Because component durations vary with discretization thresholds (active moving difference spans −5.10 s to −6.80 s and stationary difference spans +3.10 s to +5.20 s across linear velocity thresholds 0.03–0.08 m/s in our sensitivity analysis, Supplementary Material Section S5), we report these as coarse-grained observational latencies rather than asserting an invariant physical mechanism.

3. **Comparison with Spatial Caching in Static 2D Geometry (F vs. O)**: In both D1 and D2, FailMem (F) and Spatial Cache (O) selected identical topological routes. Total distance differences (+0.6% in D1, −0.9% in D2) and total duration differences (+0.7% in D1, +0.9% in D2) remained under 1%. In the evaluated static 2D hallway layout, specific implementation, and run sample ($n = 3$), no additional routing benefit of F over O was observed; this does not imply general lack of benefit across arbitrary domains or statistical equivalence. Notably, dynamic recovery upon obstacle removal is not exclusive to FailMem; spatial costmap caching (O) likewise updates its internal occupancy state and unblocks routes upon acquiring new sensory observations.

5.3 Feasibility Check for Action-Conditioned Navigation (H_1)

To investigate whether action configurations (aligned a_{aligned} vs. doorpost boundary-biased oblique a_{oblique}) create executability differences in an unobstructed doorway, a fixed 4-run feasibility check was conducted.

Table 4 summarizes the feasibility trial outcomes. Nav2 returned `SUCCEEDED` (status code 4) in all 4/4

Table 2: Condition-Level Navigation Performance Across 30 Gazebo Simulation Runs ($n = 3$ per condition, mean $\pm$ sample std dev, $ddof = 1$).

Scenario	Method	n	Actual Route	Dead-Ends	Decision Dist (m)	Decision Time (s)	Total Dist (m)	Total Time (s)	Replay Audit Pass
D0	R	3	Path_A	0.0 ± 0.0	6.16 ± 0.04	51.5 ± 1.7	6.16 ± 0.04	51.5 ± 1.7	3/3 (100%)
D0	O	3	Path_A	0.0 ± 0.0	6.20 ± 0.01	50.0 ± 0.8	6.20 ± 0.01	50.0 ± 0.8	3/3 (100%)
D0	F	3	Path_A	0.0 ± 0.0	6.20 ± 0.03	49.9 ± 0.5	6.20 ± 0.03	49.9 ± 0.5	3/3 (100%)
D1	R	3	Path_A_then_Path_B	1.0 ± 0.0	10.82 ± 0.07	82.1 ± 4.4	17.22 ± 0.24	141.0 ± 5.9	3/3 (100%)
D1	O	3	Path_B	0.0 ± 0.0	7.55 ± 0.05	50.0 ± 2.0	13.95 ± 0.07	108.7 ± 2.7	3/3 (100%)
D1	F	3	Path_B	0.0 ± 0.0	7.54 ± 0.00	48.2 ± 1.0	14.03 ± 0.25	109.5 ± 1.6	3/3 (100%)
D2	R	3	Path_A	0.0 ± 0.0	5.92 ± 0.04	49.6 ± 1.6	16.34 ± 0.03	148.2 ± 3.2	3/3 (100%)
D2	O	3	Path_A	0.0 ± 0.0	5.94 ± 0.07	52.0 ± 1.7	16.40 ± 0.24	149.8 ± 0.6	3/3 (100%)
D2	F	3	Path_A	0.0 ± 0.0	5.89 ± 0.02	51.1 ± 1.0	16.25 ± 0.10	151.1 ± 7.8	3/3 (100%)
D2	M1	3	Path_B	0.0 ± 0.0	7.89 ± 0.02	51.3 ± 2.0	18.25 ± 0.09	148.8 ± 2.4	3/3 (100%)

Table 3: Key Pairwise Contrasts Between Methods Across Scenarios D1 and D2.

Scenario	Contrast	Metric	A	B	Diff $(A - B)$	Rel (%)
D1	F vs. R	Dist (m)	14.03	17.22	-3.19	-18.5%
D1	F vs. R	Time (s)	109.50	141.00	-31.50	-22.3%
D1	O vs. R	Dist (m)	13.95	17.22	-3.27	-19.0%
D1	O vs. R	Time (s)	108.70	141.00	-32.30	-22.9%
D1	F vs. O	Dist (m)	14.03	13.95	+0.08	+0.6%
D1	F vs. O	Time (s)	109.50	108.70	+0.80	+0.7%
D2	F vs. $M1$	Dist (m)	16.25	18.25	-2.00	-11.0%
D2	F vs. $M1$	Time (s)	151.10	148.83	+2.27	+1.5%
D2	F vs. O	Dist (m)	16.25	16.40	-0.15	-0.9%
D2	F vs. O	Time (s)	151.10	149.77	+1.33	+0.9%

runs, and strict physical arrival was verified in 3/4 runs (`H1_aligned_run1` exceeded the angular velocity halt threshold during the stability window: $0.1068 > 0.08$ rad/s). Nav2's `DWBLocalPlanner` navigated the passage without planner abortion across all runs.

Decision & Conclusion: *Nav2 returned success in 4/4 runs, with strict physical arrival verified in 3/4 runs; the candidate scenario did not establish the expected executability divergence. Exploration for this hypothesis was therefore stopped per protocol, which does not constitute a refutation of the general action-conditioned failure memory hypothesis.*

6 Threats to Validity & Limitations

1. **Exploratory Sample Size**: The 30-run comparative pilot ($n = 3$ per condition) serves as an exploratory demonstration of mechanism functioning and auditability; it does not constitute large-sample asymptotic hypothesis testing.

2. **Simulation Fidelity**: Evaluated in Gazebo 11 simulation with nominal sensor models. Physical robot hardware deployments with real-world sensor dropout, dynamic obstacles, and adversarial conditions are subject to future research.

3. **Conceptual Extensions ($F2, O+$)**: Action-profiled failure memory ($F2$) and backoff spatial caching ($O+$) are formalized as conceptual models in project documentation but have not yet been evaluated in simulation.

4. **Within-Session Phase-Level Scope**: Memory evaluation is restricted to sequential phases within a single continuous experimental execution session. Cross-task persistence across operating system restarts, long-term storage indexing, and multi-day operation remain unverified.

7 Conclusion

This paper presented FailMem, an event-driven failure memory architecture for autonomous mobile robots. By binding execution failures to immutable observation bundles and providing an independent replay audit pipeline, FailMem establishes rigorous standards for reproducible robotics evaluation. Our empirical findings demonstrate that historical information eliminates redundant dead-end exploration relative to reactive retry, and dynamic state updates (available in both FailMem and spatial caching) prevent permanent detour traps upon environmental recovery. In the evaluated layout and run sample, no additional routing benefit of explicit failure memory over spatial caching was observed. These findings provide an auditable, reproducible benchmark and empirical stopping rationale to guide future research on action-conditioned and multi-agent memory systems.

Acknowledgment

[Funding Grant Placeholder: This work was supported by Grant Placeholder `#XXXX-XXXX` from Funding Agency Placeholder. Compute resources were provided by Institution Placeholder.]

Table 4: Exploratory Feasibility Results for Action-Conditioned Navigation Hypothesis H_1 ($n = 4$).

Run Name	Profile	Target Goal	Perception	Nav2 Status	Nav2 Time (s)	Settling (s)	Stability (s)	Total Sim (s)	Strict Arrival
H1_aligned_run1	act_aligned	[1.50, 1.20]	FREE (23)	SUCCEEDED (4)	13.4	3.0	2.4	18.8	**False** ($\omega_{\text{ang}} : 0.107 > 0.08$)
H1_aligned_run2	act_aligned	[1.50, 1.20]	FREE (22)	SUCCEEDED (4)	13.4	3.0	2.4	18.8	**True**
H1_oblique_run1	act_oblique	[0.50, 0.88]	FREE (23)	SUCCEEDED (4)	11.1	3.0	2.3	16.4	**True**
H1_oblique_run2	act_oblique	[0.50, 0.88]	FREE (22)	SUCCEEDED (4)	11.0	3.0	2.3	16.3	**True**

References

[1] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2718–2725, IEEE, 2020.

[2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[3] D. V. Lu, D. Hershberger, and W. D. Smart, "Layered costmaps for context-sensitive navigation," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 709–715, IEEE, 2014.

[4] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.

[5] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, "Voxblox: Incremental 3D euclidean signed distance fields for on-robot exploration," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1366–1373, IEEE, 2017.

[6] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 8634–8652, 2023.

[7] Z. Liu, A. Bahety, and S. Song, "REFLECT: Summarizing robot experiences for failure explanation and correction," in *Conference on Robot Learning (CoRL)*, vol. 229 of *Proceedings of Machine Learning Research*, pp. 3468–3484, PMLR, 2023.

[8] R. E. Fikes and N. J. Nilsson, "STRIPS: A new approach to the application of theorem proving to problem solving," *Artificial Intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.

[9] R. Simmons, R. Goodwin, K. Z. Haigh, S. Koenig, and J. O'Sullivan, "The Xavier mobile robot: Autonomous navigation in human environments," *IEEE Robotics & Automation Magazine*, vol. 5, no. 1, pp. 34–42, 1998.

[10] F. Ingrand and M. Ghallab, "Deliberation for autonomous robots: A survey," *Artificial Intelligence*, vol. 247, pp. 10–44, 2017.

[11] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *Conference on Robot Learning (CoRL)*, vol. 205 of *Proceedings of Machine Learning Research*, pp. 1769–1782, PMLR, 2023.

[12] D. Berenson, S. S. Srinivasa, and J. Kuffner, "A robot path planning framework for adaptive experience reuse," *The International Journal of Robotics Research*, vol. 31, no. 10, pp. 1237–1254, 2012.

[13] M. Phillips, B. J. Cohen, S. Chitta, and M. Likhachev, "E-Graphs: Bootstrapping planning with experience graphs," *The International Journal of Robotics Research*, vol. 31, no. 10, pp. 1159–1175, 2012.

[14] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A search algorithm for propositional satisfiability," *IEEE Transactions on Computers*, vol. 48, no. 5, pp. 506–521, 1999.